

Mock Exam

Computer Architecture

Name: _____

Matr. no.: _____

Number of pages: 21

Available time: 90 Minuten

Allowed resources: None

Please check if you have received **all** sheets and fill in your name and your matriculation number on **all** sheets (also on the additional sheets you have received). Sheets without this information will not be evaluated.

Question	Number of points	
	Possible	Aquired
1	15	
2	15	
3	15	
4	15	
5	15	
6	15	
Total	90	

Achieving **45** points is sufficient to pass the exam.

WITHDRAWING FROM AN EXAMINATION

In case of illness, you must supply proof of your illness by submitting a medical report to the Examinations Office. Please note that the medical examination must be done at the latest on the same day of the missed exam. In case of illness while writing the exam please contact the supervisory staff, inform them about your illness and immediately see your doctor. The medical certificate must be submitted latest 3 days after the medical examination. More information's: https://www.tf.uni-freiburg.de/studies/exams/withdrawing_exam.html

CHEATING/DISTURBING IN EXAMINATIONS

A student who disrupts the orderly proceedings of an examination will be excluded from the remainder of the exam by the respective examiners or invigilators. In such a case, the written exam of the student in question will be graded as 'nicht bestanden' (5.0, fail) on the grounds of cheating. In severe cases, the Board of Examiners will exclude the student from further examinations.

General information through this exam:

- whenever we ask for RISC-V code, we do not care about the exact instruction name. If you remember it, fine. If not, give it a reasonable name!
- Unless mentioned otherwise, we only consider 32-bit RISC-V.
- Unless specified otherwise, if we ask you to write programs, you do need to save registers.
- Every exercise gives the same number of points.

Exercise 1 (2+5+5+3 points)

We consider the following C structure:

```
struct {  
    int x;  
    int y;  
} POINT;
```

Assume that there are no padding bits, that x is before y , and that there is no extra alignment condition. Therefore, the memory representation is $|x|y|$.

- a) What is the size of the structure in words?
- b) How do you access x and y assuming that the pointer to `POINT` is in `a0`?
- c) We want to sort coordinates in an array

```
int compare (struct POINT * a, struct POINT * b)  
{  
    if (a->x < b->x)  
        return -1;  
    if (a->x == b->x) {  
        if (a->y < b->y) return -1;  
        else if (a->y == b->y) return 0;  
        else return 1;  
    }  
    return 1;  
}
```

We provide you an extract of the RISC-V code. Complete the holes wherever needed and annotate it to match the C code.

```
compare:                                     # @compare  
    addi    sp, sp, ----  
    sw      ra, 28(sp)  
    sw      s0, 24(sp)  
    addi    s0, sp, 32  
    sw      a0, -16(s0)  
    sw      a1, -20(s0)  
    lw      a0, -16(s0)
```

```

        mv      a2, a0
        lw      a3, --- (a0)
        lw      a4, --- (a1)
        li      a0, -1
        blt     a3, a4, ---
        li      a0, 1
        bne     a3, a4, ---
        lw      a2, --- (a2)
        lw      a1, --- (a1)
        li      a0, -1
        blt     a2, a1, ----
        xor     a0, a2, a1
        snez    a0, a0

end:
        lw      ra, 28(sp)
        lw      s0, 24(sp)
        addi    sp, sp, ----
        ret

```

d) We want to compare an array of coordinates to having two array of coordinates:

```

struct {
    int* xs;
    int* ys;
} COORDINATES;

```

We have the following compare function:

```

int compare (COORDINATES a, int i, int j)
{
    if (a.xs[i] < a.xs[j])
        return -1;
    if (a.xs[i] == a.xs[j]) {
        if (a.ys[i] < a.ys[j]) return -1;
        else if (a.ys[i] == a.ys[j]) return 0;
    }
    return 1;
}

```

Annotate and complete it:

```

compare:                                     # @compare
        slli    a2, a2, ----
        add     a4, a0, a2
        lw      a4, 0(a4)

```

```
slli    a3, a3, ----
add     a0, a0, a3
lw      a5, ___ (a0)
li      a0, -1
blt     a4, a5, -----
bne     a4, a5, -----
add     a2, a2, a1
lw      a2, 0(a2)
add     a1, a1, a3
lw      a1, 0(a1)
blt     a2, a1, -----
bne     a2, a1, -----
li      a0, 0
ret
.LBB0_5:
li      a0, 1
.LBB0_6:
ret
```

e) Why is there no operation on the stack pointer `sp`?

(<http://www.catb.org/esr/structure-packing/> is *the* reference for everything about padding and alignment).

Exercise 2 (2+2+11 points)

We consider a cache with an associativity of 2. We consider a very simple cache with 2 lines and you go from the real address to the cache line by going modulo 2.

Whenever we read an address in memory, we actually fetch a cache line composed of $2 * \lfloor \frac{n}{2} \rfloor$ (rounded down) and $2 * \lfloor \frac{n}{2} \rfloor + 1$ (rounded down + 1). For example, if you fetch the address 4, you will fetch 4 and 5. If you fetch the address 9, you will fetch 8 and 9.

When overwrite we overwrite the *least-recently used* entry.

We will use the following kind of table:

timestamp cache line	1				2			
	0	1	2	3	0	1	2	3
0	• ⁺	10	•	20	• ⁺	30	•	20
1		1	• ⁺	21	• ⁺	31	• ⁺	21

With the convention that • indicates that the content is valid and ⁺ indicates that this is the newest read / write. In the cache we write the *address* of the content – we do not care about the actual content in this exercise.

- Explain the idea of fetching multiple addresses at once and give an example (text!) of a program where this is useful.
- Why is fetching one row not as slow as fetching each address?
- We consider that the program is doing the following things:

```

read 1
read 2
another thread writes 2
read 1
read 6
read 7

```

Fill the following table:

line \ time	0		1	2	3	4	5	6	7
	0	2							
0	0	2							
1	1	3							

Name: _____

Matriculation number: _____

Additional pages for exercise 2 – page 1

Exercise 3 (6+3+3+3 points)

We consider two different CPUs. The newer one has two kind of new instructions, SIMD-1 and SIMD-2. Your original program (Prog 1) has 120 instructions. You write two new programs that uses in addition only SIMD-1 instructions (Prog 2) and both SIMD-1 and SIMD-2 (Prog 3).

The new one at 4 GHz.

	Normal	SIMD-1	SIMD-2
CPI	1	2	3
Prog 1	120		
Prog 2	50	10	
Prog 3	20	1	3

- Calculate the average CPI for all three programs.
- Calculate the CPU time for the new processor for all three programs if you execute it 10^9 times.
- While debugging the program you use GDB. Does setting a breakpoint changes the instructions executed by the CPU?
- While debugging the program you use GDB. What happens in the CPU when a breakpoint is reached?

Your solution to exercise 3:

Name: _____

Matriculation number: _____

11

Your solution to exercise 3 (Continued):

Your solution to exercise 3 (Continued):

Exercise 4 (2+2+2+1+1+1+2+2+2 points)

In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
100 ps	150 ps	350 ps	300 ps	200 ps

Also, assume that the instructions executed by the processor are distributed as follows over the average execution time (e.g. on average 45% of the executed instructions are ALU/Logic instructions):

ALU/Logic	Jump/Branch	Load	Store
45%	20%	20%	15%

- What is the clock cycle time in a pipelined processor?
- What is the clock cycle time in a non-pipelined processor?
- What is the total latency of an **add** instruction (no memory usage) in a pipelined processor?
- Considering only the stages that are needed to run the instruction, what is the total latency of an **add** instruction (no memory usage) in a non-pipelined processor?
- If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split?
- If you split the stage as answered in the previous question, what is the new clock cycle time of the processor?
- Assuming there are no stalls or hazards, how often is the data memory used in percent?
- Assuming there are no stalls or hazards, how often is the write-register port of the “Registers” unit in percent?
- Actually the reason why the EX step is so long is the support for division. The CPU is changed such that: (i) the EX step now takes only 200 ps, and (ii) a division takes *two cycles*. Considering that 5% of the instructions are divisions, what is the average CPI?

(Based on an exercise from “Computer Organization and Design” by David A. Patterson, John L. Hennessy)

Your solution to exercise 4:

Your solution to exercise 4 (Continued):

Name: _____

Matriculation number: _____

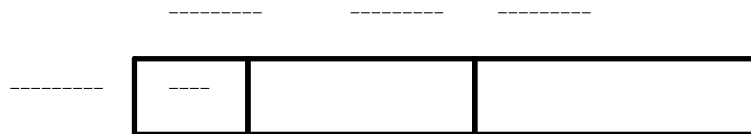
15

Your solution to exercise 4 (Continued):

Exercise 5 (3+2+2+2+2+2+2 points)

We consider a cache of size 16 KiB (2^{14} Bytes) – this is the size of the content in cache, it does not include headers. In each block, we have *four* (4) words.

- a) Complete the following graphic of a cache line and indicate the size of the first block:



[-1 per wrong]

- b) We consider a direct mapping
- 1) What is the offset?
 - 2) What is the index size?
 - 3) What is the tag size?
- c) We consider a two-way mapping
- 1) What is the offset?
 - 2) What is the index size?
 - 3) What is the tag size?

Your solution to exercise 5:

Name: _____

Matriculation number: _____

17

Your solution to exercise 5 (Continued):

Exercise 6 (5+5+5 points)

To limit the size of our ALU we use GRS bits (i.e., we do not have to keep more bits than the GRS bits in intermediate results) and we use round to even to avoid propagation errors. We use bfloat8 (1 sign bit, 3 fraction bits, 4 exponent bits, exponent bias 7).

Calculate:

a)

$$1.011_2 \cdot 2^7 - 1.111_2 \cdot 2^3$$

b)

$$1.011_2 \cdot 2^{-4} - 1.000_2 \cdot 2^{-4}$$

c)

$$1.011_2 \cdot 2^{-1} - 1.101_2 \cdot 2^{-4}$$

Your solution to exercise 6:

Name: _____

Matriculation number: _____

19

Your solution to exercise 6 (Continued):

Your solution to exercise 6 (Continued):

Name: _____ *Matriculation number:* _____ 21

Additional pages for all exercises – page 1